第2章练习题：习题集2.8　2.10　2.21　2.22　2.34　2.35

下周交第1章作业和第2章作业

每次作业布置以后的第二周交作业

第2章　线性表

2.1　线性表的逻辑结构

2.2　线性表的顺序表示和实现

2.3　线性表的链式表示和实现

1. 单链表如何实现———建表和输出

例：用单链表结构来存放26个英文字母组成的线性表（a，b，c，…，z）,请写出C语言程序。

实现思路：**先开辟头指针，然后陆续为每个结点开辟存储空间并及时赋值，后继结点的地址要提前送给前面的指针。**

先挖“坑”,后种“萝卜”！

将全局变量及函数提前说明：

```
#include<stdio.h>
#include<stdlib.h>
typedef struct node{
char data;
struct node *next;
}node;
```

```
node *p,*q,*head;        //一般需要3个指针变量
int n ;                  // 数据元素的个数
in  m=sizeof(node);  /*结构类型定义好之后，每个node类型的长度就固定了，m求一次即可*/
```

```
void build( )      //字母链表的生成。要一个个慢慢链入
{ int i;
head=(node* malloc(m);     //m=sizeof(node) 前面已求出
p=head;
for( i=1; i<26; i++)        //因尾结点要特殊处理，故i≠26
{ p->data=i+'a'-1;          // 第一个结点值为字符a
p->next=(node*)malloc(m); //为后继结点“挖坑”！
p=p->next;  }               //让指针变量P指向后一个结点
p->data=i+'a'-1;            //最后一个元素要单独处理
p->next=NULL ;}             //单链表尾结点的指针域要置空!
```

新手特别容易忘记！！

```
void display()          //字母链表的输出
{
  p=head;                                   sum=0;
  while (p)    //当指针不空时循环（仅限于无头结点的情况)
   {
    printf("%c",p->data);
    p=p->next;                              //让指针不断“顺藤摸瓜”
   }                                        sum++;
}
```

讨论：要统计链表中数据元素的个数，该如何改写？

2. 单链表的修改(或读取)

缺点：想寻找单链表中第i个元素，只能从头指针开始逐一查询（顺藤摸瓜），无法随机存取 。

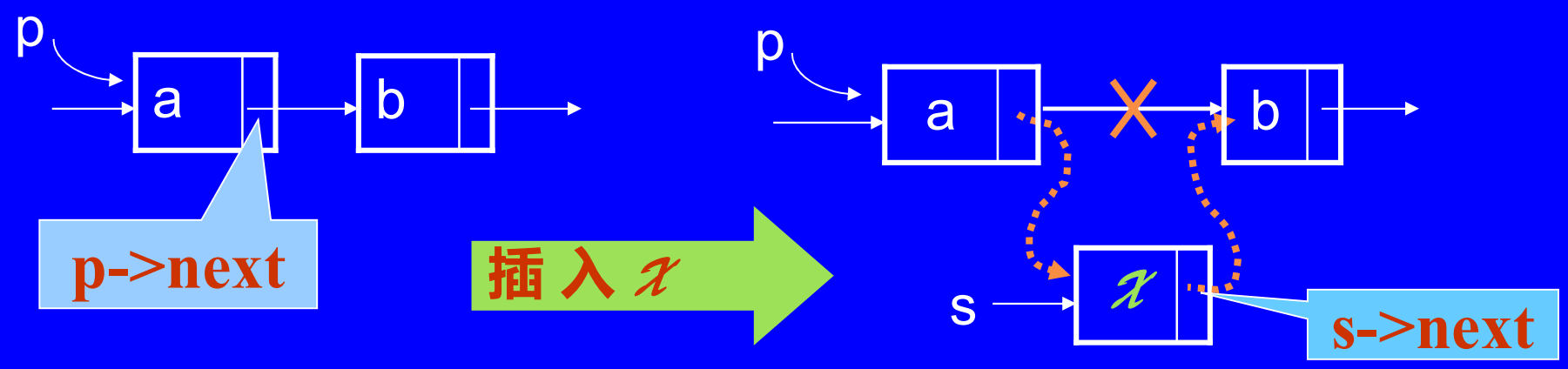
思路：要修改第i个数据元素，必须从头指针起一直找到该结点的指针p，然后才能：p->data=new_value 。

读取第i个数据元素的核心语句是：

```
Status GetElem_L(LinkList L, int i, ElemType &e)
{p=L->next; j=1;     //假设是带有头结点的链表
while(p&&j<i){p=p->next; ++j;}
if(!p||j>i)return ERROR;
e=p->data;
return OK;}
```

3. 单链表的插入

在链表中插入一个元素x的示意图如下：



链表插入的核心语句：

Step 1： s->next=p->next;
Step 2： p->next=s ；

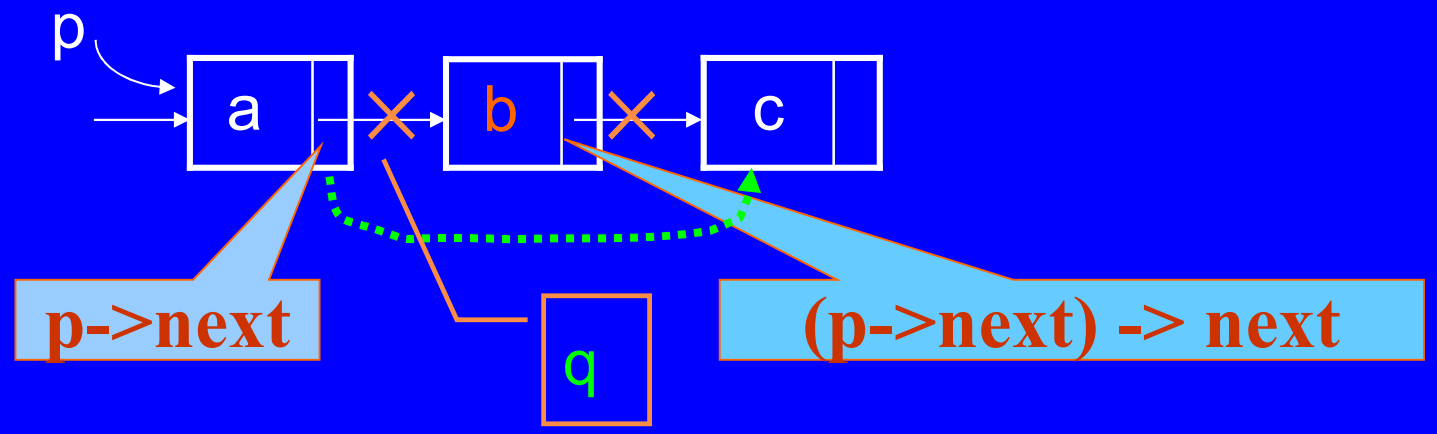
x 结点的生成方式：

```
S=(node*)malloc(m);
S->data=x;
S->next=p->next
```

思考：Step1和2能互换么？

4. 单链表的删除

在链表中删除某元素b的示意图如下：



删除动作的核心语句（要借助辅助指针变量q）：

```
q = p->next;          //首先保存b的指针，靠它才能找到c;
p->next=q->next;      //将a、c两结点相连，淘汰b结点;
free(q)；             //彻底释放b结点空间
```

思考： 省略free(q)语句行不行?

讨论： 单链表只能查找结点的直接后继，若想查找结点的直接前驱，该如何设计？

答：只要把单链表再多开一个指针域即可(例如用*next和*prior)。

prior	data	next

这种链表称为双向链表。其特点是可以双向查找表中结点。

双向链表在非线性结构（如树结构）中将大量使用。

5. 链表的运算效率分析

时间效率分析

1. 查找

因线性链表只能顺序存取，即在查找时要从头指针找起，查找的时间复杂度为 O(n)。

2. 插入和删除

因线性链表不需要移动元素，只要修改指针，一般情况下时间复杂度为 O(1)。

但是，如果要在单链表中进行前插或删除操作，因为要从头查找前驱结点，所耗时间复杂度将是 O(n)。

填空：

在n个结点的单链表中要删除已知结点*P，需找到它的 **前驱结点的地址** ，其时间复杂度为 **O(n)** 。

空间效率分析

链表中每个结点都要增加一个指针空间，相当于总共增加了n 个整型变量，空间复杂度为 O(n)。

6. 应用举例

教材例子略，完全可以自学。

两个链表的归并（**教材P31例**）

一元多项式的计算　（**教材P39–43**）

另举三例 →

微软亚洲研究院03年来汉招聘的笔试题

例1：试用C或类C语言编写一个高效算法，将一循环单链表就地逆置。

操作前：（$a_1, a_2, \ldots a_{i-1}, a_i, a_{i+1}, \ldots, a_n$）

操作后：（$a_n, \ldots a_{i+1}, a_i, a_{i-1}, \ldots, a_2, a_1$）

分析：要想让a_n指向a_{n-1},……, a_2指向a_1，一般有两种算法：

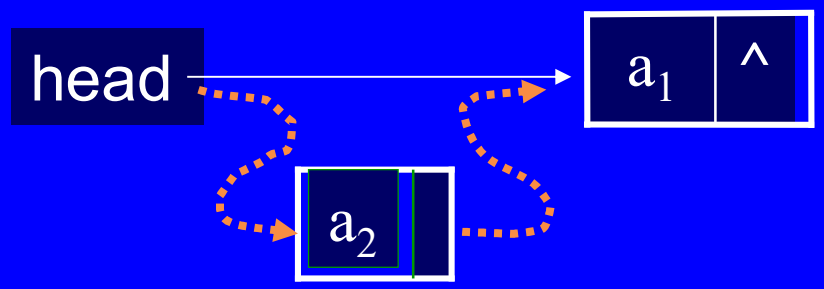
①替换法：扫描a_1……a_n，将每个a_{i-1}的指针域送入a_{i+1}的指针域。

思路：后继变前驱

②插入法：扫描a_1……a_n，将每个a_i插入到链表首部即可。

思路：头部变尾部

实际上是链栈的概念



插入法的关键语句如下：

```
p=L;  q=NULL;  L=NULL;   //无头结点

while(p)
{
   q = p->next;
   p->next = L;
   L=p;
   p=q;
}
```

问：

双向链表的插入和删除操作与单链表有何区别？

静态链表的操作与动态链表有何不同？

例2：在双向链表中如何实现插入和删除运算？

单链表中查找只能从前往后，而不能从后往前查。为了查找方便，提高查找速度，可以在结点上增加一个指针域，用来存结点的直接前驱，这样的链表，称为双向链表。其结点的结构为：

prior data next

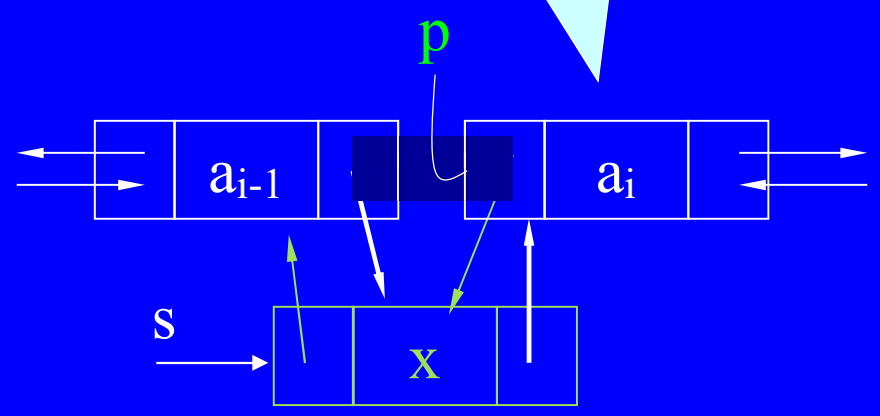
双向链表类型的定义如下：

```
typedef   struct  DuLNode{
      ElemType          data;          //数据域
     struct DuLNode  *prior;          //前驱指针域
     struct DuLNode  *next;           //后继指针域
} DuLNode , *DuLinkList ;
```

双向链表的插入操作：

注意：要修改双向指针！

设p已指向第 i 元素，请在第 i 元素前插入元素 x



指针域的变化：

① a_{i-1}的后继从 a_i (指针是p)变为 x（指针是s) :

s->next = p ； p->prior->next = s ;

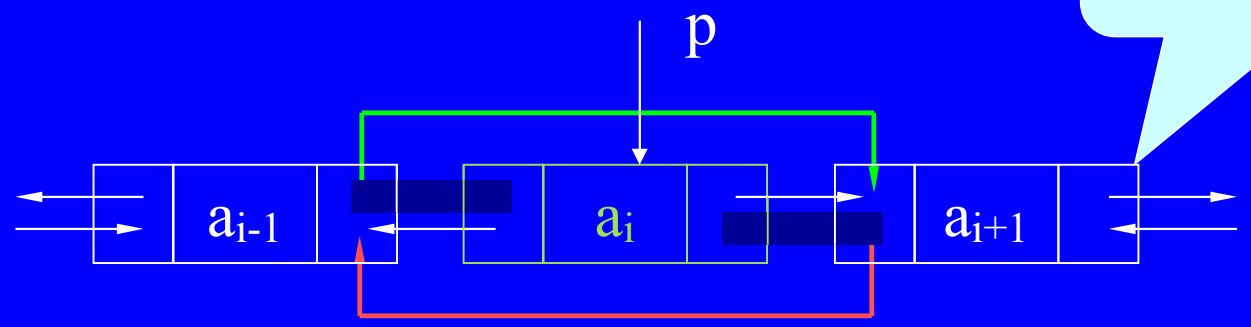
② a_i 的前驱从 a_{i-1} (指针是**p->prior**)变为 x (指针是s);

s->prior = p ->prior ; p->prior = s ;

双向链表的删除操作：

设p指向第 i 个元素，删除第 i 个 元素

注意：要修改双向指针！



指针域的变化：

后继方向：a_{i-1}的后继由 a_i (指针p)变为 a_{i+1}(指针 p ->next);

p ->prior->next = p->next ;

前驱方向：a_{i+1} 的前驱由 a_i (指针p)变为a_{i-1} (指针 p -> prior);

p->next->prior = p ->prior ;

往年微软、百度、腾讯等来招聘，都反复出了此题！

基本要求：试用C语言编写一个高效算法，将一循环单链表就地逆置。（带或不带头结点均可）

高级要求：自命题， 运用链表知识自编一个实用小软件，并撰写较为规范的文档。

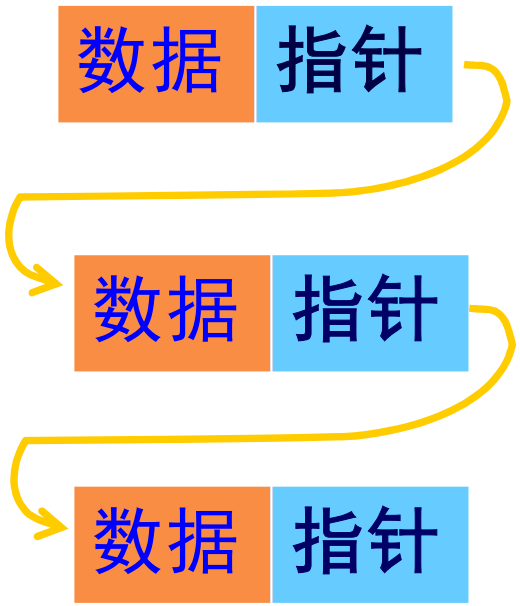
讨论： 若某种高级语言没有指针类型，能否实现链式存储和运算？如何实现？

答：能！虽然链表通常用动态级联方式存储，但也可以用一片连续空间（一维数组）实现链式存储，这种方式称为静态链表。

具体实现方法：定义一个结构型数组（每个元素都含有数据域和指示域），就可以完全描述链表，指示域就相当于动态链表中的指针。

指示域也常称为“游标”

动态链表样式：



静态链表样式：

数组中每个元素都至少有两个分量，属于结构型数组。

常用于无指针类型的高级语言中。

例 3：一线性表 S = (ZHAO, QIAN, SUN, LI, ZHOU, WU)，用静态链表如何表示？——教材P32例题

i	data	cur
0	头结点	1
1	ZHAO	3
2	LI	5
3	QIAN	6
4	WU	0
5	ZHOU	4
6	SUN	2
…	……	……
1000		

说明1：假设S为SLinkList型变量，则S[MAXSIZE] 为一个静态链表；S[0].cur则表示第1个结点在数组中的位置。

说明2：如果数组的第i个分量表示链表的第k个结点，则：

S[i].data表示第k个结点的数据；

S[i].cur 表示第k+1个结点(即k的直接后继)的位置。

说明3：静态链表的插入与删除操作与普通链表一样，不需要移动元素，只需修改指示器就可以了。

例如：在线性表 S = (ZHAO, QIAN, SUN, LI, ZHOU, WU)的QIAN, SUN之间插入新元素LIU，可以这样实现：

i	data	cur
0	头结点	1
1	ZHAO	3
2	LI	5
3	QIAN	7
4	WU	0
5	ZHOU	4
6	SUN	2
7	LIU	6
1000	…	…

Step1:将QIAN的游标值存入LIU的游标中：

```
S[7].cur = S[3].cur;
```

Step2:将QIAN的游标换为新元素LIU的下标：

```
S[3].cur = 7
```

本章小结

1. 线性结构(包括表、栈、队、数组）的定义和特点：
仅一个首、尾结点，其余元素仅一个直接前驱和一个直接后继。

2. 线性表
- 逻辑结构：“一对一” 或 “1:1”
- 存储结构：顺序、链式
- 运　算：修改、插入、删除[查找和排序另述]

3.顺序存储
- 特征：逻辑上相邻，物理上也相邻；
- 优点：随机查找修改快　O(1)
- 缺点：插入、删除慢　O(n)

改进方案：链表存储结构

4.链式存储

- **特征：逻辑上相邻，物理上未必相邻；**
- **优点：插入、删除快　O(1)**
- **缺点：随机查找修改慢 O(n)**

5.几种特殊链表的特点：

静态链表的特点：不用指针也能实现链式存储和运算

循环链表的特点：从任一结点出发均可找到表中其他结点

双向链表的特点：可方便找到任一结点的前驱

补概念（自学）：

- 指针的物理意义
- &和*的区别
- 循环单链表高效逆置的算法

讨论： 顺序存储和链式存储的区别和优缺点？

顺序存储时，逻辑上相邻的数据元素，其物理存放地址也相邻。顺序存储的优点是存储密度大，存储空间利用率高；缺点是插入或删除元素时不方便。

链式存储时，相邻数据元素可随意存放，但所占存储空间分两部分，一部分存放结点值，另一部分存放表示结点间关系的指针。链式存储的优点是插入或删除元素时很方便，使用灵活。缺点是存储密度小，存储空间利用率低。

顺序表适宜于做**查找**这样的静态操作；
链表宜于做**插入、删除**这样的动态操作。
若**线性表的长度变化不大**，且其主要操作是查找，则采用顺序表；
若**线性表的长度变化较大**，且其主要操作是插入、删除操作，则采用链表。

讨论1：什么是指针？指针的作用？

指针—即变量的内存地址。指针主要功能有二：

①提供了一种快速访问数组单元的途径；

②使C语言函数可以修改其调用的参数。

讨论2：与指针有关的符号&和*之间有何区别？

&---指针操作符（单目），返回操作数地址；

*---运算符（单目），是对&的补充，返回位于这个地址内的变量之值。

例：q=*m意即“q取地址m中的值”。如果数值100存储在内存地址2000中，而这一地址又存在m中，则q=(2000)=100

而 q=&m 则意味着 q=2000

& 类似于“直接寻址”，* 类似于“间接寻址”

第2章练习题：习题集**2.8　2.10　2.21　2.22　2.34　2.35**

下周交第2章作业